



CSC 3402

Fundamental

s of Search

Department of Computer Science
School of Natural Sciences
University of Zambia

Solving Problems by Searching

- When the correct action is not immediately clear, an agent needs to plan ahead by considering a sequence of actions to reach a goal
- Such agents are called **problem-solving agents**, and their **computational process is known as Search**
 - Problem-solving agents use **atomic representations** where states are considered as wholes without internal structure, focusing only on the current state as a unit
- **Types of Search Algorithms**

1. Informed algorithms: The agent has an estimate of how far it is from the goal (e.g., using a heuristic)

2. Uninformed algorithms: The agent has no estimate of the distance to the goal

Problem-Solving Agents

- Consider the following problem scenario (*from Chap 3 of the 4th ed. of the prescribed book*)
 - An agent is in Romania in the city of Arad
 - Has a nonrefundable ticket to fly out of Bucharest the following day
 - There are 3 roads to get to Bucharest from Arad: one toward Sibiu, one to Timisoara, and one to Zerind
 - None of these are the goal, must decide which road to take
 - Without knowledge of Romania, the agent would have to choose randomly

- For our search problem solving we assume differently, that is, that our agents always have access to information about the world, such as a map of Romania in this case

Problem-Solving Agents

Figure 3.1

Problem-Solving Agents

- With access to information the agent can follow the **Four-Phase Problem Solving Process**
 1. **Goal Formulation:** The agent **adopts the goal of reaching Bucharest**, which limits the actions it needs to consider
 2. **Problem Formulation:** The agent **creates an abstract model of the world**, focusing on cities and travel between them
 3. **Search:** The agent **simulates possible action sequences** (routes) to find a solution that reaches the goal - if no solution exists, the agent recognizes that
 4. **Execution:** Once a solution is found, agent **executes the actions step by step**

Problem-Solving Agents

- In this topic, search algorithms are applied in simple environments: episodic, single-agent, fully observable, deterministic, static, discrete, and known environments

- **Deterministic Environments and Open-Loop Systems**

- In fully observable, deterministic, known environments, the **solution is a fixed sequence of actions**, and the agent can ignore percepts (acting as an open-loop system) since the solution guarantees reaching the goal

- **Partially Observable or Nondeterministic Environments**

- In less predictable environments, the agent may **require a branching strategy with contingency plans**, adjusting actions based on new percepts (e.g., arriving in the wrong city or encountering a road closure)

Problem-Solving Agents - Search Problems & Solutions • State Space

- The **set of all possible states** that the environment can be in
- This defines the search problem's entire scope

- **Initial State**

- The specific state **where the agent starts**

- Example: Starting in the city of Arad

- **Goal State(s)**

- A set of **one or more target states** where the agent wants to end up • An

IS-GOAL method checks if a state meets the goal criteria

Problem-Solving Agents - Search Problems & Solutions • Available Actions

- The **set of actions the agent can perform** in each state.

- For instance, from Arad, the actions are to travel to Sibiu, Timisoara, or Zerind
- This is denoted as ***ACTIONS(s)***, where ***s*** is the current state,
 - E.g. ***ACTIONS(Arad) = {ToSibiu, ToTimisoara, ToZerind}***

- **Transition Model**

- Describes the **result of applying an action** in a particular state
- For example, ***RESULT(Arad, ToZerind)*** would return the state of being in Zerind
 - Written as ***RESULT(Arad, ToZerind) = Zerind***

Problem-Solving Agents - Search Problems & Solutions

- **Action Cost Function**

- A function that **assigns a cost to each action**, usually based on performance measures

- For example, in route-finding, the cost could be distance in miles or time to complete an action
- Denoted as ***ACTION-COST***(s, a, s') which gives the numeric cost of applying action in state s to reach state s'

- **Path and Solution**

- A **path** is a **sequence of actions** leading from the **initial state** to the **goal state**.
- A **solution** is a **path that successfully leads** to the **goal**
- **Optimal Solution**: The **solution with the lowest total cost**, calculated as the sum of the costs of each action in the path

Problem-Solving Agents - Search Problems & Solutions

- **Action Cost Assumptions**

- **Assumes that action costs are additive** (total path cost is the sum of action costs) and positive, avoiding complications like negative-cost cycles
- **State Space as a Graph**
 - The **state space can be visualized as a graph** where
 - **vertices** represent **states**, and
 - **edges** between them represent **actions**
 - The map of Romania is an example of such a graph, with roads indicating possible actions in both directions

Problem-Solving Agents - Formulating Problems

- Formulating a problem, like getting to Bucharest, is an **abstract mathematical description**, not a reflection of every real-world detail (e.g., weather, traffic, or scenery)

- **Abstraction Process**

- **Abstraction** involves **removing irrelevant details** to simplify the problem. •

This ensures **that only relevant details for reaching the goal** are included

- A good problem formulation **simplifies actions without overwhelming detail** (e.g., “drive from Arad to Sibiu” instead of “move foot forward”)
- An **abstract solution** (e.g., a route from Arad to Bucharest) represents many possible detailed paths (e.g., different driving experiences such as radio on/off)

Problem-Solving Agents - Formulating Problems

- An abstraction is **valid** if an **abstract solution** can be elaborated into a detailed solution **that still reaches the goal**
- The abstraction is **useful** if the actions **can be easily carried out without**

further detailed planning, making the solution easier than the original problem

- The ability to create **useful abstractions allows agents to avoid being overwhelmed by real-world complexity**, enabling more efficient problem solving

Example Problems

- The **problem-solving approach** has been applied to various task environments, covering both **standardized** and **real-world problems**
- **Standardized Problems**
 - Used to **demonstrate or test problem-solving methods** using **concise, exact descriptions**, making them **suitable for research and algorithm performance benchmarking**

- **Real-World Problems**

- Solutions for these are applied in **practical** settings customized to **specific environments / systems** (e.g., robot navigation) and focus on **usable solutions**
- They are often **idiosyncratic** (specific and not standardized), as different systems (e.g., robots) have unique sensors and produce different data

Example Problems - Standardized

Problems • Grid World Problem

- **Structure:** A grid world consists of a 2D array of square cells where agents move from cell to cell, usually in horizontal, vertical, and sometimes diagonal directions
- **Interaction with Objects**
 - Cells can contain objects that agents interact with, such as picking up or pushing items

- Obstacles like walls prevent movement into certain cells
- **States**
 - A state specifies which objects are in which cells
 - For a vacuum world with 2 cells, there are 8 possible states (agent position and dirt presence) - a vacuum world with n cells has $2 \cdot 2^n$ states

Example Problems - Standardized

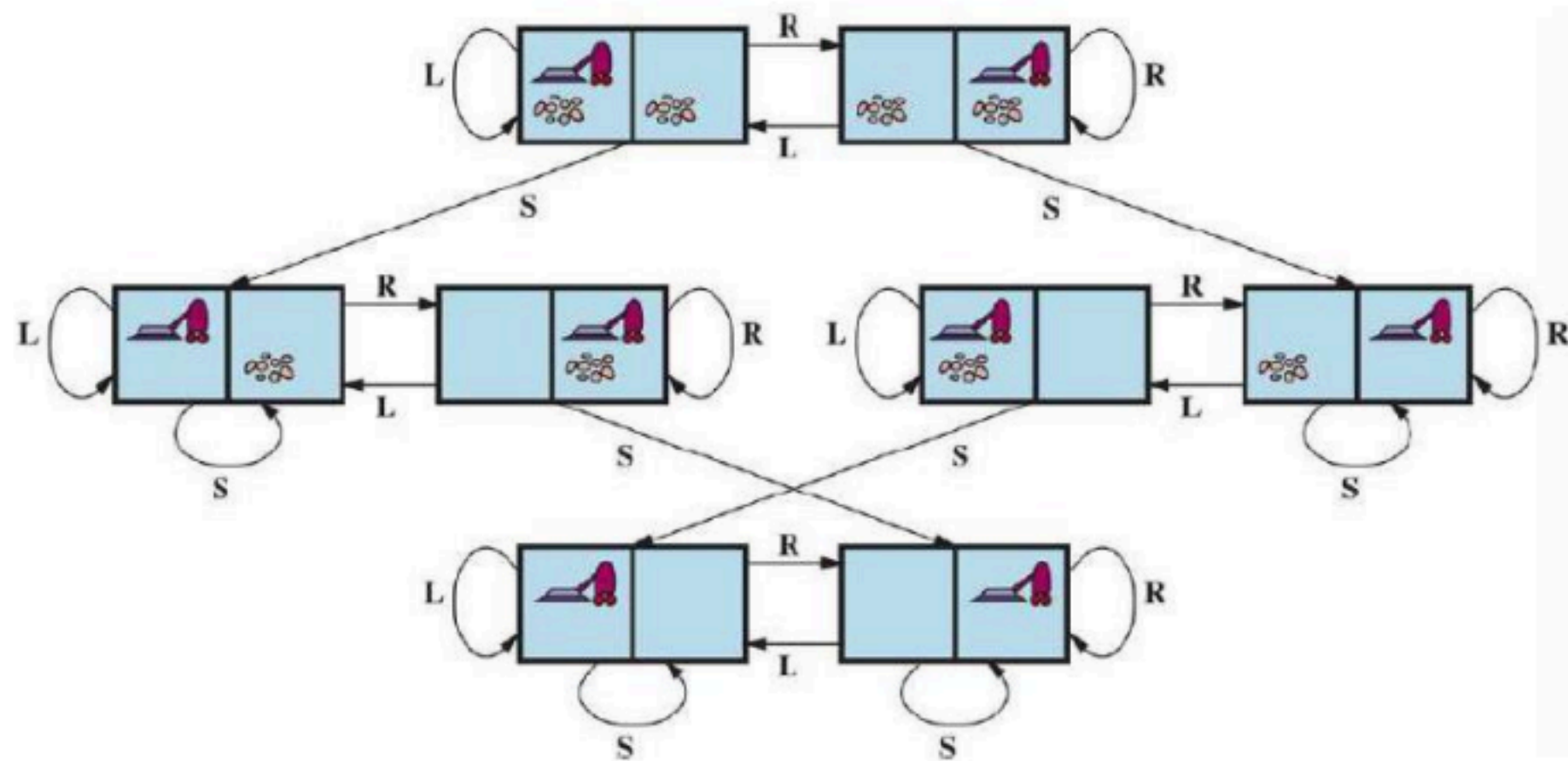
Problems • Grid World Problem

- **Initial State:** Any state can be designated as the starting point
- **Actions**
 - For a two-cell vacuum world, actions include Suck, Move Left, and Move Right

- In larger grid worlds, actions like Move Up and Move Down can be added
- **Transition Model:** Actions such as Suck remove dirt, and Forward moves the agent one cell ahead unless blocked by a wall
- **Goal States:** The goal is for all cells to be clean
- **Action Cost:** Each action has a cost of 1

Example Problems - Standardized Problems

Figure 3.2



The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = *Left*, R = *Right*, S = *Suck*.

Example Problems - Standardized

Problems • Sokoban Puzzle

- Another type of grid world
- **Objective:** The agent must push boxes scattered on the grid to designated storage locations
- **Movement Rules:** An agent can push a box if there is an empty cell behind it, but cannot push a box into another box or wall
- **State Space**
 - For n non-obstacle cells and b boxes, the state space size is $n! \times b! / (n - b)!$
 - On an 8x8 grid with 12 boxes, there are over 200

trillion states

Example Problems - Standardized

Problems • Sliding-Tile Puzzle

- **Structure:** Tiles are arranged in a grid with one or more blank spaces, allowing the tiles to slide into the blank space
- **Variants**
 - Includes the 8-puzzle (3x3 grid with 8 tiles) and the 15-puzzle (4x4 grid with 15 tiles)
 - The goal is to arrange the tiles in a specific order
- **States:** Each state specifies the location of each tile
- **Initial State:** Any arrangement of the tiles can serve as the initial state

Example Problems - Standardized

Problems • Sliding-Tile Puzzle

- **Actions:** Movement of the blank space Left, Right, Up, or Down (not all actions are applicable in edge/corner positions)
- **Transition Model:** For example, moving Left in the 8-puzzle swaps the 5 and the blank
- **Goal State:** Typically, the numbers in order
- **Action Cost:** Each action costs 1

Example Problems - Standardized

Problems • Sliding-Tile Puzzle

Example Problems - Standardized Problems

- **Infinite State Spaces Example (Knuth's Problem):** arise in tasks involving mathematical expressions, circuits, proofs, programs, and recursively defined objects
 - **Objective:** Use square root, floor, and factorial operations to reach any desired positive integer starting from 4
 - **State Space:** Positive real numbers, with infinite possible states
 - **Initial State:** The number 4
 - **Actions:** Apply square root, floor, or factorial operations
 - **Transition Model:** Defined by mathematical operations
 - **Goal State:** The desired positive integer
 - **Action Cost:** Each action costs 1

Example Problems - Real-World

Problems • Route-Finding Problems

- **Applications:** Used in systems providing driving directions, routing video streams, military operations, and airline travel planning
- **Example - Airline Travel**
 - **States:** Include current location, time, and historical flight information (fare bases, international/domestic status)
 - **Initial State:** The user's home airport
 - **Actions:** Taking a flight considering seat class, time, and transfer
 - **Transition Model:** Results in a new location and time after the flight

Example Problems - Real-World

Problems • Route-Finding Problems

- **Example** - Airline Travel continued
 - **Goal State:** A destination city (e.g., Bucharest or other complex goals like nonstop flights)
 - **Action Cost:** Includes money, waiting time, flight time, customs, seat quality, etc
 - **Complications:** Unexpected flight delays and missed connections

Example Problems - Real-World

Problems • Touring Problems

- **Description:** Involves visiting multiple locations rather than just reaching a single destination
- **Traveling Salesperson Problem (TSP):** Requires visiting every city on a map with minimal cost, with algorithms used for optimization
- **Applications:** Routing school buses, planning movements of circuit-board drills, and shop floor stocking

Example Problems - Real-World

Problems • Very-Large-Scale Integration (VLSI)

Layout Problems

- **Task:** Position millions of components and connections on a chip to minimize area and delays while maximizing yield
- **Cell Layout:** Grouping circuit components into cells and arranging them on a chip to avoid overlap

- **Channel Routing:** Finding specific routes for wires between cells
- **Complexity:** Extremely challenging but crucial for chip design

Example Problems - Real-World

Problems • Robot Navigation

- **Description:** Generalizes route-finding, where robots make their own paths instead of following distinct ones
- **Challenges:** Many-dimensional search spaces (e.g., controlling each joint of a robot), errors in sensors and motor controls, and handling partial observability
- **Advanced Techniques:** Needed to make the continuous search space finite and deal with real-world obstacles

Example Problems - Real-World

Problems • Automatic Assembly Sequencing

- **Objective:** Finding the correct order to assemble parts for objects like electric motors
- **Challenges:** Choosing the wrong order may lead to difficulties in adding parts later, making this a difficult geometrical search problem
- **Applications:** Used to optimize the assembly line to reduce manual labor

- **Protein Design**

- Involves finding amino acid sequences that fold into proteins with specific properties, such as curing diseases

Search Algorithms

- A **search algorithm** takes a **search problem as input** and **returns** either a **solution** or an indication of **failure**
- To solve the problem, the algorithm **builds a search tree over the state space graph**
- The **state space** represents **all possible states** in the world and the **actions** that **allow transitions** between them
- While the **search tree organizes paths between states** that the **algorithm explores to find a path to the goal**

Search Algorithms

- **State Space**
 - Contains **all possible states** and the **transitions (actions) between them**

- **Search Tree**
 - **Organizes paths between states**
 - Each **node** in the tree corresponds to a **state**, and the **edges** represent **actions** leading from one state to another
 - Each node has a unique path back to the root (starting state)
- **Distinction between the state space and the search tree** is crucial: while the state space contains all possible configurations of the environment, the search tree explores specific paths within this space, working toward finding a solution

Search Algorithms

- **Nodes in the Search Tree**
 - The **root node** represents the **initial state** (e.g., Arad in the Romania problem).
 - **Child nodes** (or successor nodes) are **generated by applying actions to the**

current state, using the RESULT function to determine the resulting state

- The **parent node** refers to the **node from which the child was generated**

- **Frontier**

- The **frontier** consists of **all nodes that have been generated but not yet expanded** (i.e., *their child nodes haven't been generated yet*)
- It separates the **states that have been explored** (interior) from **states that haven't been explored** (exterior)

Search Algorithms

Search Algorithms

Search Algorithms

Search Algorithms

- **Expanding Nodes**

- At each step, the **algorithm must choose which node** on the frontier **to expand**
- **Expanding a node** means **generating its child nodes based on the possible actions** from that state
 - For example, if the agent expands the node for Sibiu, the search tree now contains more unexpanded nodes representing possible further actions from Sibiu

- **Goal of Search**

- The search algorithm continues expanding nodes until it finds a path to a goal state or determines that no solution exists

Search Algorithms - Best-First Search

- **Best-first search** selects the next node to expand **based on the minimum value** of an evaluation function $f(n)$

- **Node selection:** On each iteration, the algorithm chooses the node on the frontier with the lowest $f(n)$ value
- **Goal check:** If the chosen node's state is a goal state, the algorithm returns it as the solution
- **Expansion:** If the node is not a goal state, the algorithm expands the node, generating its child nodes

Search Algorithms - Best-First Search

- **Frontier management:** Each child node is added to the frontier if it has not been reached before or re-added if it is reached with a lower path cost than previously recorded

- **Outcome:** The algorithm continues this process until it either finds a goal state or the frontier is empty, indicating failure
- **Flexibility:** By changing the $f(n)$ function, best-first search can adapt into different specific search algorithms

Search Algorithms - Best-First Search

Search Algorithms - Best-First Search

Search Algorithms - Search Data Structures •

Node representation: Each node in the search tree has four components •

STATE: The state the node corresponds to

- **PARENT:** The node that generated this node

- **ACTION:** The action applied to the parent's state to generate this node
- **PATH-COST:** The total cost from the initial state to this node
- **Path recovery:** Following the PARENT pointers from a node allows recovery of the path from the initial state to that node

Search Algorithms - Search Data Structures

- **Frontier Management:** The frontier is stored in a queue, which supports the following operations
 - **IS-EMPTY:** Checks if the frontier has no nodes
 - **POP:** Removes and returns the top node from the frontier
 - **TOP:** Returns (without removing) the top node from the frontier

- **ADD:** Inserts a node into the frontier

Search Algorithms - Search Data

Structures • Queue Types

1. **Priority Queue:** Pops the **node with the minimum cost** (used in **Best-First Search**)
 2. **FIFO Queue (First-In-First-Out):** Pops the **node added first** (used in **Breadth First Search**)
 3. **LIFO Queue (Last-In-First-Out):** Pops the most recently added node (used in **Depth-First Search**)
- **State Tracking:** Reached **states are stored in a lookup table** (e.g., hash table) with states as keys and nodes as values

Search Algorithms - Redundant Paths

- **Redundant Paths**

- Paths in the search tree **that lead to the same state but through different sequences of actions**, often resulting in unnecessary exploration

- **Cycle (Loopy Path)**

- A special type of redundant path where the **search returns to a previously visited state**, potentially making the search tree infinite

- **Example**

- In a 10x10 grid world, although each square can be reached in 9 moves or fewer, there can be over a million redundant paths of the same length, significantly slowing down the search

Search Algorithms - Redundant Paths

- **Approaches to Handle Redundant Paths:**

1. **Remember all Reached States:** Track previously visited states to detect and eliminate redundant paths, keeping only the best path to each state

- Preferred for state spaces with many redundant paths and when memory allows

2. **Ignore Redundant Paths:** Suitable for problems where paths rarely overlap, allowing for reduced memory usage but potentially slower searches due to re exploring states

3. **Check for Cycles:** Detect cycles by examining the parent chain of a node

- This can either be a full check (eliminating all cycles) or a partial check (eliminating short cycles), balancing memory usage and search efficiency

Search Algorithms - Measuring Solving

Performance • Performance Evaluation Criteria for search

algorithms

1. **Completeness:** Ensures that the algorithm **will find a solution** if one exists and **correctly report failure if none exists**
2. **Cost Optimality:** Determines whether the algorithm finds the **solution with the lowest path cost** among all possible solutions
3. **Time Complexity:** Measures **how long the algorithm takes to find a solution**, either in real-time (seconds) or by the number of states/actions considered
4. **Space Complexity:** Assesses the **amount of memory required** to perform the search

Search Algorithms - Measuring Solving

Performance • Completeness Considerations

- **In finite state spaces**, completeness is achieved by systematically exploring all reachable states and avoiding cycles
- **In infinite state spaces**, algorithms must be careful not to follow infinite paths without ever reaching a goal
 - Systematic exploration is needed to ensure that every state connected to the initial state is eventually reached
- **Example of systematic exploration:** On an infinite grid, a spiral search pattern can cover all cells step-by-step, ensuring completeness

Search Algorithms - Measuring Solving Performance

- **Time and Space Complexity:** These are evaluated with respect to problem difficulty, which can be measured by:

- **Depth (d):** Number of actions in an optimal solution
- **Maximum Path Length (m):** Maximum number of actions in any path
- **Branching Factor (b):** Number of successors a node can have, indicating the number of paths that need to be considered

Uninformed Search Strategies

- **Uninformed Search Algorithms**

- Uninformed search algorithms **operate without any additional information** about how close a given state is to the goal
- **Example:** An agent in Arad with the goal of reaching Bucharest would be equally likely to choose a path toward Zerind or Sibiu without knowing which is closer to the goal
- These algorithms **lack knowledge of the problem domain** and must **rely solely on the structure of the search tree**

- **Informed Search Algorithms (Contrast)**

- Informed search algorithms **have additional information**, such as the distance between cities, which **helps guide the search more efficiently**
- **Example:** An informed agent would know that Sibiu is closer to Bucharest than Zerind, making Sibiu a more likely step on the shortest path

Uninformed Search Strategies - Breadth-First

Search • Breadth-First Search Overview

- **Strategy:** Expands the root node first, then all its successors, followed by their successors, and so on
- **Systematic Approach:** This method is systematic and complete, even in infinite state spaces
- **Evaluation Function:** Can be seen as a specific case of best-first search, where the evaluation function $f(n)$ is the depth of the node

Uninformed Search Strategies - Breadth-First


```

function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure

```

```

function UNIFORM-COST-SEARCH(problem) returns a solution node, or failure
  return BEST-FIRST-SEARCH(problem, PATH-COST)

```

Search

Breadth-first search and uniform-cost search algorithms.

Uninformed Search Strategies - Breadth-First

Search • Breadth-First Efficiency Enhancements

- **Queue Type:** Uses a **first-in-first-out (FIFO)** queue instead of a priority queue, which is **faster and orders nodes correctly** based on depth
- **State Tracking:** Uses a set of states (rather than a mapping from states to nodes) to **avoid redundant paths**, as once a state is reached, it can't be improved upon
- **Early Goal Test:** Checks whether a node is a solution as soon as it is **generated**, rather than after it's removed from the queue

Uninformed Search Strategies - Breadth-First

Search • Optimality and Completeness

- **Solution Optimality:** Always finds the solution with the minimal number of actions, making it cost-optimal for problems where all actions have the same cost
- **Completeness:** Guarantees finding a solution if one exists

• Time and Space Complexity

- **Exponential Growth:** In a uniform tree with branching factor **b**, the number of nodes generated grows exponentially as $O(b^d)$, where **d** is the depth of the solution.
- **Memory Requirement:** All nodes must remain in memory, leading to both **time and space complexities of $O(b^d)$**
 - **Example:** A search with **b=10** and **d=10** would take less than 3 hours but require 10 terabytes of memory; at **d=14**, it would take 3.5 years even with infinite memory

Uninformed Search Strategies - Breadth-First

Search • Limitations

- **Memory Intensive:** Breadth-first search **requires significant memory**, making it impractical for deep searches
- **Exponential Complexity:** The exponential growth of time and space usage makes breadth-first search **unsuitable for large or complex problems** without additional optimizations or heuristics

Uninformed Search Strategies - Uniform-Cost

Search • Uniform-Cost Search Overview

- **Purpose:** Used when actions have different costs, **selecting the path with the lowest total cost**
- **Comparison to Breadth-First Search:** While breadth-first search explores nodes in order of uniform depth, uniform-cost search explores nodes based on uniform path cost

- **Algorithm Process**

- Can be implemented as best-first search with the evaluation function set to path cost
- **Node Expansion:** Nodes are expanded in order of increasing path cost from the root to the current node
- **Goal Test:** The algorithm tests for goal states only when expanding a node, not when generating it

Uninformed Search Strategies - Uniform-Cost Search

Uninformed Search Strategies - Uniform-Cost

Search • Example Scenario

- **Path Selection:** In a scenario where the agent is moving from Sibiu to Bucharest, the

algorithm expands nodes based on the lowest cost, ultimately finding the least expensive path to the goal

- **Goal Detection:** If the goal were checked upon node generation rather than expansion, the algorithm might return a higher-cost path

- **Complexity**

- **Factors:** Complexity is characterized by C^* (the cost of the optimal solution) and ϵ (a lower bound on the cost of each action)

- **Worst-Case Complexity:** The worst-case time and space complexity is $O(b^{1+\lceil C^*/\epsilon \rceil})$, which can exceed b^d due to the exploration of many low-cost actions before encountering higher cost, potentially more effective actions

Uninformed Search Strategies - Uniform-Cost

Search • Key Properties

- **Completeness:** Uniform-cost **search is complete**, meaning it will find a solution if one exists
- **Cost-Optimality:** It is cost-optimal, ensuring **the first solution found is the least costly** compared to any other node in the frontier
- **Systematic Exploration:** The algorithm **explores paths systematically in order of increasing cost**, avoiding infinite loops (assuming all action costs are greater than $\epsilon > 0$)

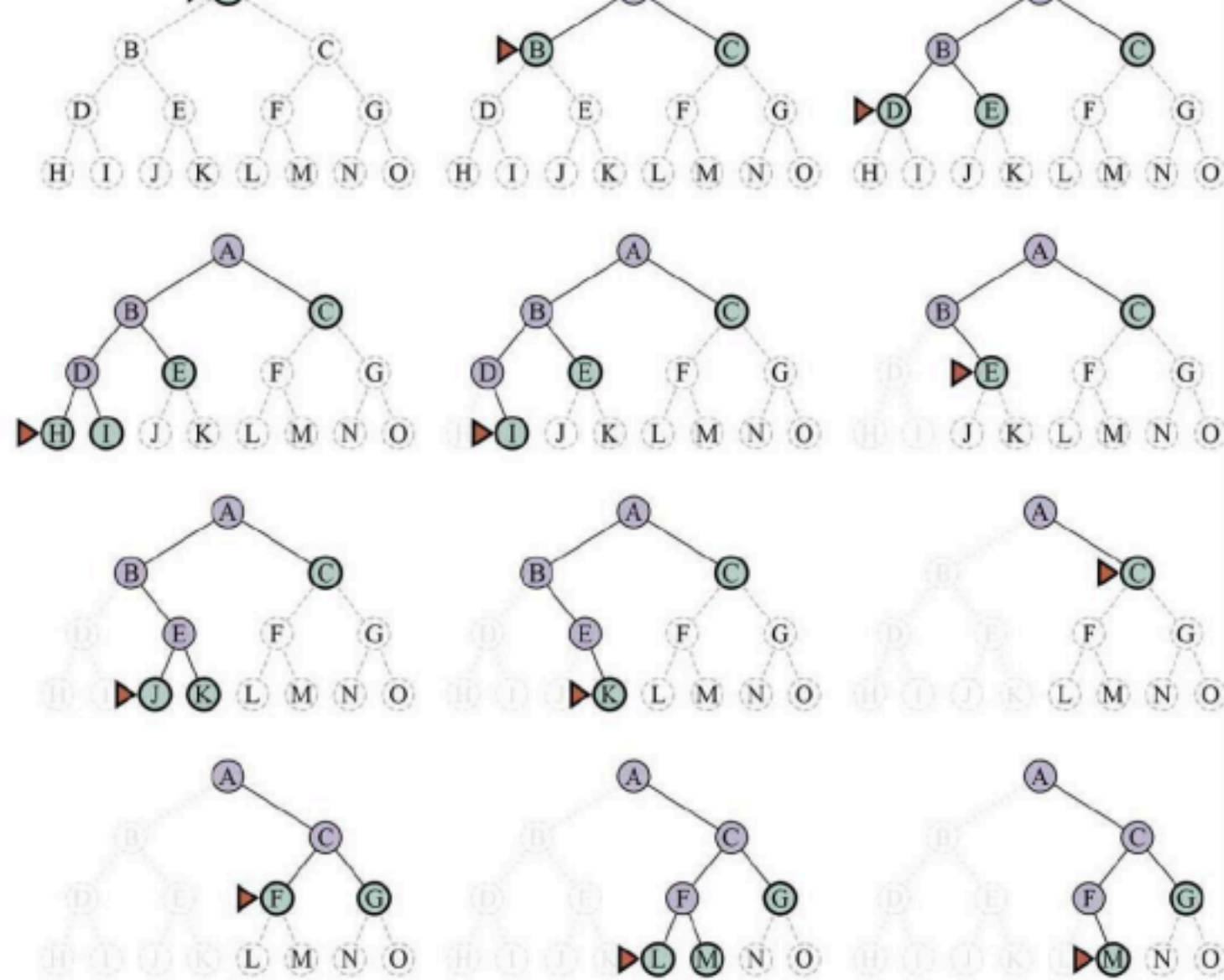
Uninformed Search Strategies - Depth-First

Search • Depth-First Search (DFS) Overview

- **Strategy:** DFS **expands the deepest node** in the frontier first
- **Implementation:** Typically **implemented as a tree-like search**, not a graph search, which means it does not maintain a table of reached states

- **Search Process**

- **Progression:** DFS **immediately explores to the deepest level of the search tree** and then "backs up" to the next deepest node with unexpanded successors
- **Solution Quality:** DFS is **not cost-optimal**; it returns the first solution it finds, which may not be the cheapest



A dozen steps (left to right, top to bottom) in the progress of a depth-first search on a binary tree from start state A to goal M. The frontier is in green, with a triangle marking the node to be expanded next. Previously expanded nodes are lavender, and potential future nodes have faint dashed lines. Expanded nodes with no descendants in the frontier (very faint lines) can be discarded.

d Search Strategies

-

Depth-First Search

Uninformed Search Strategies - Depth-First

Search • Efficiency and Completeness

- **Finite State Spaces (Trees):** DFS is **efficient and complete** in finite tree-shaped state spaces.
- **Acyclic State Spaces:** DFS **may expand the same state multiple times** via different paths but will eventually explore the entire space.

- **Cyclic State Spaces:** DFS can get stuck in infinite loops, so some implementations include cycle-checking.
- **Infinite State Spaces:** DFS is incomplete because it can get stuck in infinite paths even without cycles.

Uninformed Search Strategies - Depth-First

Search • Memory Usage

- **Memory Efficiency:** DFS requires much less memory than breadth-first search (BFS)
 - It **does not store a reached table**, and the frontier is small, akin to the radius of an expanding sphere
- **Memory Complexity:** For a finite tree-shaped state space, DFS has a time complexity proportional to the number of states and a memory complexity of $O(b^m)$, where **b** is the branching factor and **m** is the maximum depth
- **Memory Comparison:** Problems requiring exabytes (1,000,000 TB) of memory with BFS

can be handled with kilobytes using DFS

Uninformed Search Strategies - Depth-First

Search • Backtracking Search Variant

- **Memory Optimization:** Uses even less memory by generating one successor at a time and modifying the current state directly.
- **Memory Complexity:** Reduces memory requirements to one state description and a path of $O(m)$ actions.
- **Cycle Checking:** Can efficiently check for cyclic paths in $O(1)$ time if needed.
- **Application:** Critical for large state descriptions, such as robotic assembly. • **Use**

Cases of DFS

- DFS is widely used in AI areas such as **constraint satisfaction**, **propositional satisfiability**, and **logic programming** due to its minimal memory requirements.

Uninformed Search Strategies - Depth-Limited & Iterative Deepening Search

- **Depth-Limited Search (DLS)**

- **Limits the depth of depth-first search to a predefined level, l** , treating nodes at depth l as if they had no successors.
- **Time Complexity:** $O(b^l)$.
- **Space Complexity:** $O(bl)$.
- **Limitation:** If l is too small, **the search might fail to find a solution**, making it incomplete.

- **Cycle Elimination in DLS**

- DFS as a tree-like search can waste time on redundant paths; however, **cycles can be**

handled by checking a few links up the parent chain or by using a depth limit.

Uninformed Search Strategies - Depth-Limited & Iterative Deepening Search

- **Choosing a Good Depth Limit**

- A good depth limit might be known **based on problem knowledge** (e.g., diameter of the state-space graph), but in many problems, a good depth limit is unknown until the problem is solved.

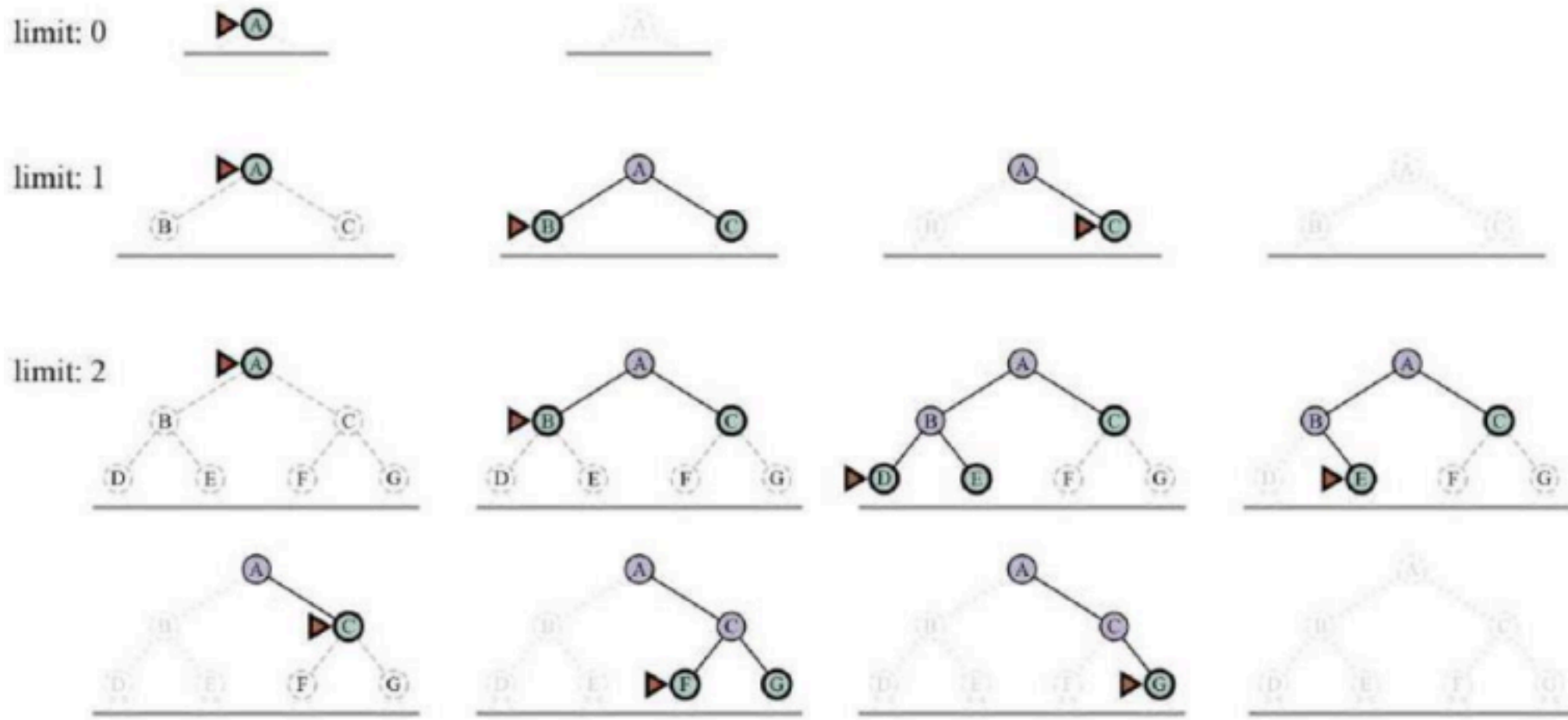
Uninformed
Search
Strategies - Depth
Limited

& Iterative Deepenin g Search Uninformed Search Strategies - Depth-Limited & Iterative Deepening Search

- **Iterative Deepening Search (IDS)**
 - Solves the problem of selecting an appropriate depth limit by **trying all possible depths: first 0, then 1, then 2, and so on.**
 - **Memory Requirements:** $O(bd)$ when there is a solution, or $O(bm)$ in finite state spaces with no solution.

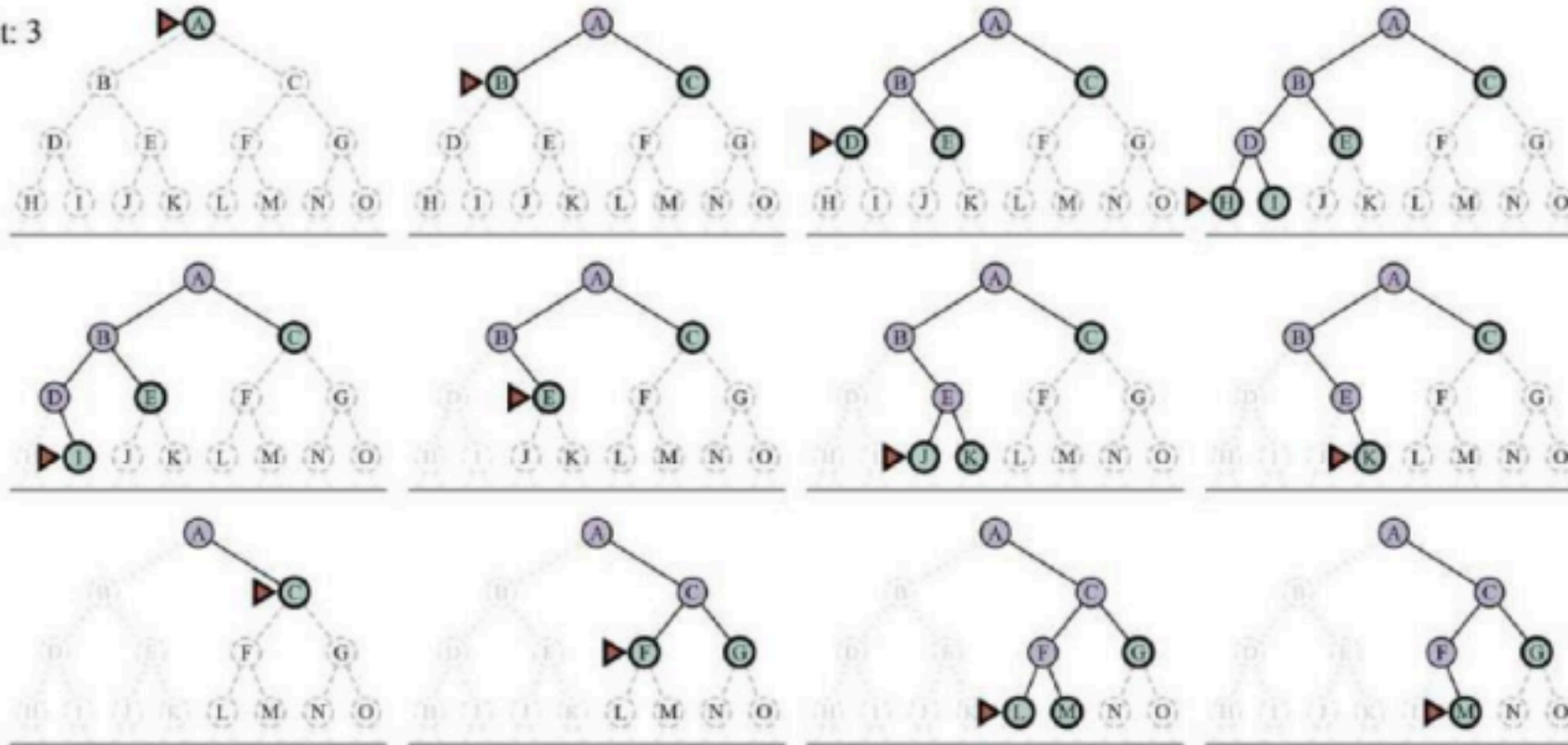
- **Time Complexity:** $O(b^d)$.
- **Optimality:** Like breadth-first search (BFS), IDS is **optimal for problems where all actions have the same cost**.
- **Completeness:** IDS is **complete for finite acyclic state spaces** or any finite state space when cycles are checked.

Uninformed Search Strategies - Depth-Limited & Iterative Deepening Search



Uninformed Search Strategies - Depth-Limited & Iterative Deepening Search

limit: 3



Four iterations of iterative deepening search for goal M on a binary tree, with the depth limit varying from 0 to 3. Note the interior nodes form a single path. The triangle marks the node to expand next; green nodes with dark outlines are on the frontier; the very faint nodes provably can't be part of a solution with this depth limit.

Uninformed Search Strategies - Depth-Limited & Iterative Deepening

Search

- **IDS Comparison to BFS**

- IDS repeats upper levels multiple times but has the same asymptotic time complexity as BFS: $O(b^d)$. •

Example comparison: If **$b=10$** and **$d=5$** , IDS generates **123,450** nodes compared to **111,110** for BFS. •

Hybrid Approach

- **Combines BFS** until memory is nearly full, followed by IDS from all nodes in the frontier, as a way to manage memory more efficiently.
- **Preferred Use:** IDS is generally the preferred uninformed search method when the search space is larger than available memory and the solution depth is unknown.

Uninformed Search Strategies - Bidirectional

Search • Bidirectional Search

- **Simultaneously searches forward** from the initial state **and backward** from the goal state(s).
- The motivation is the **reduced complexity**: $b^{d/2} + b^{d/2}$ is much smaller than b^d , offering a **significant speedup**.

• Tracking Frontiers

- Requires **two frontiers** (one for each direction) and **two tables of reached states**.
- The search is **successful when the two frontiers meet**.

• Backward Reasoning

- **Requires the ability to reason backward**, ensuring that if state s_2 is a successor of s_1 in the forward direction, then s_1 must be a successor of s_2 in the backward direction.

Uninformed Search Strategies - Bidirectional

Search

```

function BiBF-SEARCH( $problem_F, f_F, problem_B, f_B$ ) returns a solution node, or failure
   $node_F \leftarrow \text{NODE}(problem_F.INITIAL)$  // Node for a start state
   $node_B \leftarrow \text{NODE}(problem_B.INITIAL)$  // Node for a goal state
   $frontier_F \leftarrow$  a priority queue ordered by  $f_F$ , with  $node_F$  as an element
   $frontier_B \leftarrow$  a priority queue ordered by  $f_B$ , with  $node_B$  as an element
   $reached_F \leftarrow$  a lookup table, with one key  $node_F.STATE$  and value  $node_F$ 
   $reached_B \leftarrow$  a lookup table, with one key  $node_B.STATE$  and value  $node_B$ 
   $solution \leftarrow failure$ 
  while not TERMINATED( $solution, frontier_F, frontier_B$ ) do
    if  $f_F(\text{TOP}(frontier_F)) < f_B(\text{TOP}(frontier_B))$  then
       $solution \leftarrow \text{PROCEED}(F, problem_F, frontier_F, reached_F, reached_B, solution)$ 
    else  $solution \leftarrow \text{PROCEED}(B, problem_B, frontier_B, reached_B, reached_F, solution)$ 
  return  $solution$ 

```

Uninformed Search Strategies - Bidirectional

Search

```
function PROCEED(dir, problem, frontier, reached, reached2, solution) returns a solution
    // Expand node on frontier; check against the other frontier in reached2.
    // The variable “dir” is the direction: either F for forward or B for backward.
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
        s ← child.STATE
        if s not in reached or PATH-COST(child) < PATH-COST(reached[s]) then
            reached[s] ← child
            add child to frontier
            if s is in reached2 then
                solution2 ← JOIN-NODES(dir, child, reached2[s]))
                if PATH-COST(solution2) < PATH-COST(solution) then
                    solution ← solution2
    return solution
```

Bidirectional best-first search keeps two frontiers and two tables of reached states. When a path in one frontier reaches a state that was also reached in the other half of the search, the two paths are joined (by the function JOIN-NODES) to form a solution. The first solution we get is not guaranteed to be the best; the function TERMINATED determines when to stop looking for new solutions.

Uninformed Search Strategies - Bidirectional

Search • Different Versions

- Bidirectional search has **many variants**, similar to unidirectional search algorithms.
- **Bidirectional Best-First Search**
 - Expands the node with the **minimum value of the evaluation function**, across either frontier.
 - With **path cost as the evaluation function**, it results in bidirectional uniform-cost search, which expands no node with a cost greater than $C^*/2$, where C^* is the cost of the optimal path.
- **Optimality Considerations**
 - The **first solution found will be optimal** if the evaluation function is **based on path cost**.
 - For other evaluation functions, it's necessary to track and potentially update the best solution found until no better solution is possible.

Uninformed Search Strategies - Comparing Uninformed Search Algorithms

- **Evaluation Criteria**

- Uninformed search algorithms are **compared based on four key evaluation criteria: completeness, cost optimality, time complexity, and space complexity.**

- **Tree-like Search Versions**

- The comparison focuses on tree-like search versions that do not check for repeated states.

- **Graph Searches**

- For graph searches (which do check for repeated states), depth-first search becomes complete for finite state spaces.
- The space and time complexities for graph searches are limited by the size of the state space, specifically the number of vertices and edges (**$|V|$ and $|E|$**).

Uninformed Search Strategies - Comparing Uninformed Search Algorithms

- The **key difference in graph searches is the completeness of depth-first search** and the **bounded complexities** due to the finite state space.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Evaluation of search algorithms. b is the branching factor; m is the maximum depth of the search tree; d is the depth of the shallowest solution, or is m when there is no solution; ℓ is the depth limit. Superscript caveats are as follows: ¹ complete if b is finite, and the state space either has a solution or is finite. ² complete if all action costs are $\geq \epsilon > 0$; ³ cost-optimal if action costs are all identical; ⁴ if both directions are breadth-first or uniform-cost.

Informed (Heuristic) Search Strategies

- **Uses domain-specific hints (heuristics) to find solutions more efficiently than uninformed search strategies**
 - Uses a **Heuristic Function ($h(n)$)** to represent the estimated cost of the cheapest path from the

state at node n to a goal state

- **Example - Route-Finding Problems**

- The heuristic could be the straight-line distance (Euclidean distance) between the current state and the goal state

- **Purpose of Heuristics**

- Heuristics guide the search towards the goal by **providing an estimate of how close the current state is to the goal**

Informed Search Strategies - Greedy Best-First

Search • Greedy Best-First Search

- A form of **best-first search that expands the node with the lowest heuristic value $h(n)$ first**, prioritizing nodes that seem closest to the goal
- **Evaluation function:** $f(n) = h(n)$

- **Heuristic Example - Route-Finding Problems**

- Uses the straight-line distance heuristic (h_{SLD}), which estimates the distance to the goal, such as the straight-line distance to Bucharest

- **Heuristic's Role**

- Heuristics like h_{SLD} **require world knowledge**, as they cannot be computed directly from problem descriptions (e.g., actions and results)

Informed Search Strategies - Greedy Best-First Search

Informed Search Strategies - Greedy Best-First Search

Informed Search Strategies - Greedy Best-First

Search

Informed Search Strategies - Greedy Best-First

Search • Greedy Search in Romania Example

- Expands nodes that appear closest to Bucharest, starting from Arad. It expands Sibiu, then Fagaras, and finally reaches Bucharest.
- This path is suboptimal, as it is longer than the optimal path (by 32 miles)

• Performance

- **Complete in finite state spaces** but not in infinite ones
- **Worst-case time and space complexity is $O(|V|)$** (size of the state space), but with a good heuristic, complexity can be reduced to $O(bm)$ for certain problems

• Greedy Nature

- It **can lead to suboptimal solutions** as it focuses on immediate proximity to the goal, rather than overall cost-optimal paths